루프에서 그래프로

같은 계약, 실행 구조만 바뀐다

반복되는 실패는 로그를 따라 규칙 처방 가지로 자란다

> 루프 킷의 결과(인계본)를 받아 **같은 계약**으로 그래프를 짓고 **Studio 에서 돌린다.** 주제는 여행 정보 웹페이지 그대로다.
> 그래프에는 이미 **처방 가지 3개**가 자라 있다 — 알아본 실패를 LLM 재수집보다 먼저 규칙으로 받는 길이다.

`graph_starter_kit` · 인계본은 베트남 5일 예시 (본인 루프 결과로 바꿔도 된다) · **약 60분** = STEP 1~5 · 실행 · 검증
· 실패 기록에 가지 후보가 있으면 **STEP 6 약 20분** 더

그래프는 루프가 못 하던 세 가지를 더한다 — 대가도 세 가지다

더하는 것

병렬 — 도시 3곳을 한 걸음에 나란히 조사한다

되돌아가기 · 부분 재실행 — 페이지 단계에서 수집으로 **간선 하나**로 돌아간다. 노드 하나부터 **Fork** 로 다시 돈다

원인별 길 — 알아본 실패는 LLM 재수집보다 먼저 **규칙 처방 가지**로 간다. 루프는 전부 "한 바퀴 더"였다

치르는 대가

상태를 선언한다 — 루프는 변수를 그냥 공유했다

노드끼리 서로 모른다 — 넘길 것은 상태의 칸으로만 넘긴다

`if` **문이 데이터가 된다** — 라우터는 상태 값만 읽고 다음 노드 이름만 고른다

그래프는 루프의 상위 개념이 아니다. 계약과 검사기는 그대로 두고 **실행 구조만** 바꾼다 — 그래서 마지막에 두 판을 **같은 검사기**로 잰다. 세 가지가 필요 없으면 루프가 낫다. 노드 14개만으로도 코드가 2~3배 복잡해진다.

개요

구간	내가 하는 일	진행 신호
준비 · STEP 1~5	인계본 7개를 넣고 `STEP N 진행해줘` → 결정 13개를 고른다. **왜 그게 권장인지** 읽는다	`check_graph_readiness.py` → `[READY]` exit 0
실행	`langgraph dev` → Studio에 `{}` → Submit → 게이트 세 번 답하기 · 브라우저로 결과 보기	게이트 B에서 `done` 두 번 → END
검증	`python scripts/verify_all.py` · 루프판 비교표 채우기	exit 0
STEP 6 조건부	`python scripts/failure_report.py` 의 ③ · ③-2에 후보가 있을 때만 가지 하나를 더한다	시험지 빨간불 → 초록불 → 회귀 실패 0

STEP 6은 순서상 마지막일 뿐 필수 단계가 아님. 별도의 문제 없이 보고서가 `STEP 6 은 하지 않아도 됩니다`를 찍으면 그것으로 끝이다. replay로 돌린 기본 실행은 대개 그렇다 — 킷에 이미 있는 가지와 재수집이 대다수의 문제들에 대처한다.

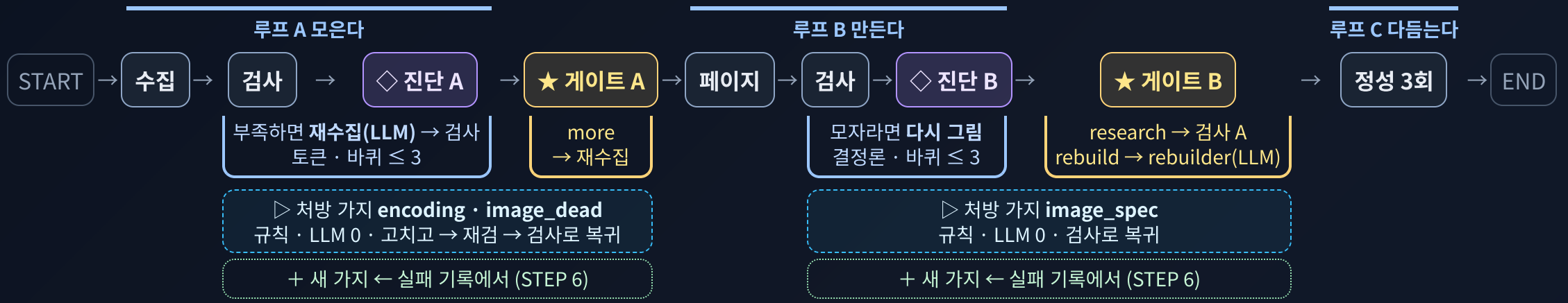
LangGraph 개념 — 노드는 바뀐 칸만 돌려주고, 라우터는 상태만 읽는다

개념	한 줄 정의	이 킷에서는
State	그래프 전체가 공유하는 작업 기억	`country` `cities` `info_verdict` `failure_kind` … 35칸 한 덩어리
Node	상태를 읽고 **변경분**을 돌려주는 함수	`collect_city` — 도시 하나의 명소를 모아 `{"cities": [city]}` 를 돌려준다
Edge	다음 실행 위치를 잇는 제어선	`build_page → check_page`
Conditional Edge	상태를 읽고 **경로를 고르는** 라우터	`failure_kind` 와 `info_verdict` 를 보고 처방 가지 · 재수집 · 게이트 중 하나
Reducer	여러 변경분을 기존 상태와 **합치는 규칙**	도시 3곳이 같은 걸음에 쓰는 `cities` 를 이름으로 합친다

> **한 걸음이 도는 방식** — 상태를 읽는다 → 노드가 돈다 → **변경분**을 돌려준다 → reducer 가 합친다 → 라우터가 결과를 읽는다 → 다음 노드를 고른다. **한 걸음에 나란히 돈 갈래가 모두 끝나야 다음 걸음이 시작된다** — Studio 에서 볼 때 이 규칙이 중요하다.

실행 모드 둘 — **live** 는 `claude -p` 로 실제 웹을 조사한다. **replay** 는 루프가 모은 `graph/research_data.json` 을 다시 써서 10초 안에 돈다.

지도 ① — 루프 골격 위에 진단이 서고, 처방 가지는 진단 옆에서 자란다



루프 골격 14

루프 킷의 일이 옮겨 온 길. **늘지도 줄지도 않는다.** 안쪽 순환(재수집 · 다시 그림)도 루프에서 그대로 왔다.

◇ 진단 2

검사기 바로 뒤에서 **매번** 돈다. 검사기는 "기준을 채웠나", 진단은 "무엇이 터졌나".

▷ 처방 가지 3 × 2

알아본 실패만 받는다. 그 실패가 나면 켜지고, 안 나면 진단만 지난다. **새 가지가 자라는 자리도 여기다.**

수집 = `collect_*` · 검사 = `check_*` · 진단 = `diagnose_*` · 게이트 = `human_gate_*` · 페이지 = `build_page` · 정성 = `quality_pass` ⇄ `check_quality`

한 실행 따라가기 — replay 기본 실행에서 진단은 매번 켜지고 가지는 켜지지 않는다

지도 ▸ **루프 A** · 수집 → 검사 → ◇진단 → 재수집 → 검사 | `run_once.py --mode replay --auto proceed` · inject 없음

걸음	노드	돌려준 변경분 (바뀐 칸만)	라우터가 읽은 값 → 다음
1~3	`read_request` → `collect_country` // `select_cities` → `collect_city` ×3	입력 칸 · 국가 소개 · `cities += 1` ×3 (reducer 가 합친다)	정적 간선 2개 = 병렬 · `Send` ×3 · 합류 → `check_information`
4	`check_information`	`info_verdict="SHORT"` · 부족 9곳 · `loop_a_iter=1`	정적 간선 하나 → `diagnose_info`
5	`diagnose_info`	`failure_kind=""` — 처방 가지 대상 없음	진단명 없음 → SHORT · 1/3바퀴 → `Send` → `repair_city` ×3
6	`repair_city` ×3	`cities` — 같은 도시를 이름으로 덮어쓴다	합류 → `check_information`
7	`check_information`	`info_verdict="PASS"` · `loop_a_iter=2`	→ `diagnose_info`
8	`diagnose_info`	`failure_kind=""`	진단명 없음 · PASS → `gate` → ★ `human_gate_a`

`run_once.py` 끝의 history 중 루프 A 부분(앞 4줄) — 글자는 편집 없이 옮김 · 이어서 `human_gate_a: proceed` · `check_page` · `diagnose_page` 줄이 더 찍힌다

```
check_information: SHORT · 부족 9곳 · 1바퀴
diagnose_info: 처방 가지 대상 없음 → 루프 골격 그대로
check_information: PASS · 부족 0곳 · 2바퀴
diagnose_info: 처방 가지 대상 없음 → 루프 골격 그대로
```

그래프는 이렇게 자란다 — 알아본 실패는 LLM 재수집보다 먼저 규칙 가지로 받는다

지도 ▸ ◇진단 A 뒤 | 같은 검사기 · 같은 진단 — 바뀌는 것은 진단이 보내는 곳뿐

```
처음 — 원인을 몰라도 전부 재수집(LLM)
 check_information ─▶ ◇diagnose_info ─▶ repair_* ──────────▶ check_information   토큰 · 매번 다른 결과 · 바퀴 +1

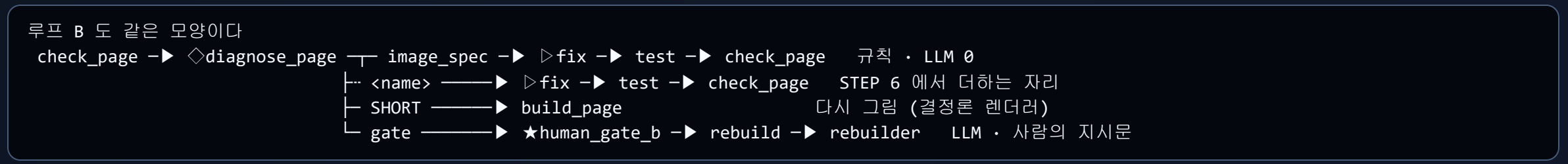
로그를 보고 가지를 연 뒤 — 알아본 실패는 규칙으로 먼저
 check_information ─▶ ◇diagnose_info ─┬─ image_dead ─▶ ▷fix ─▶ test ─▶ check_information   규칙 · LLM 0 · 바퀴 안 씀
                                      ├┄ <name> ─────▶ ▷fix ─▶ test ─▶ check_information   STEP 6 에서 더하는 자리
                                      └─ SHORT ──────▶ repair_* ─────▶ check_information   남은 것만 LLM
```

왜 `image_dead` 가 가지가 됐나 — 루프 킷 실측 (킷에 든 예시 인계본 `graph/04_RUN_EVIDENCE.md` 1-b — 직접 뽑은 인계본에는 없음): 이미지 11장이 모두 죽어 있었다. 파일은 실재했고, 모델이 주소 속 **해시 두 글자** `8/8c` 를 지어냈다. 해시는 파일 이름에서 나오는 값이라 모델이 알 방법이 없다 — 다시 모아도 또 지어낸다. 파일 이름만 뽑아 권장 형태(`Special:FilePath`)로 다시 쓰면 **같은 입력에 같은 결과**가 나온다. 그래서 규칙 가지다.

가지의 끝은 셋뿐이다 — 재검 합격이면 **같은 검사기로 다시 잰다** · 불합격이면 처방 한도(2) 안에서 한 번 더 · 한도를 다 쓰면 ★ **게이트**로 간다 (검사기를 거치지 않는다).

재수집 루프와 그래프 확장 진단 — 노드로 규격화시키면서 LLM 에 맡기는 일을 줄인다.

	재수집 `repair_*` (LLM)	규칙 처방 가지 `fix_*` → `test_*`
누가 고치나	`claude -p` 가 지적된 칸을 다시 찾아 patch 로 보낸다	등록된 규칙 코드 `travel_graph/detours/<이름>.py`
같은 입력이면	매번 다른 결과	같은 결과
비용	토큰 · 갈래마다 `claude -p` 1회 (live 에서 보통 수 분)	LLM 호출 0 (live 는 주소를 열어 보는 네트워크만)
합격 판정	검사기가 다음 바퀴에 다시 센다	같은 시험지로 재검 → 합격하면 검사기로 복귀해 다시 잰다
루프	상한 3	루프 카운트 X — 처방 한도 2 → ★ 게이트
언제	원인을 모르거나, 고칠 법이 정해지지 않은 부족	진단이 알아본 실패 — 등록된 진단명



그래프 확장 노드 예시 3개 - 실패를 규칙으로 빼고, 노드로 확장한다.

루프 골격 14 → + ◇ **진단 2** (검사기 뒤 · 매번) → **킷 배포본 22** (▷ 가지 3 × 2 — 아래 표) → **내 그래프 24** (▷ 내 가지 1 × 2 — 실패 기록에서 되풀이된 것 · STEP 6 · 후보가 있을 때만)

가지 (루프)	무엇을 잡고 어떻게 고치나	출생	재수집(LLM)에 맡겼다면
`encoding` (A)	깨진 글자 · 비ASCII 주소 → 되읽기 · 퍼센트 인코딩. 2회째 못 살린 칸은 비워 재수집에 넘긴다	**알려진 실패 축** — Windows cp949 · 현지 글자 파일명. 로그 실측 전에 미리 만들었다	검사기는 글자 **수**만 센다 — 판정이 PASS 면 재수집이 불리지도 않고 깨진 글자가 그대로 나간다
`image_dead` (A)	404 · 금지된 주소 형태 → 해시 든 주소를 권장 형태로 · 인계본의 살아 있는 사진 · (live 2회째) Commons 검색	**루프 킷 실행 로그** — 11장 전부 죽음 (예시 인계본 `04_RUN_EVIDENCE` 1-b)	해시를 알 방법이 없어 다시 모아도 또 지어낸다
`image_spec` (B)	원본 크기 사진(width 없음 · 1280 초과) · 틀 없음 · 가려진 사진 → 사진 주소를 width=1024 로 맞추고(데이터) 화면을 다시 그린다. 틀 없음 · 가려짐은 화면 코드(page.py) 문제라 못 고쳐 게이트 B 로 넘긴다	**실습생 게이트 입력** — "이미지 규격이 안 맞음"	검사기는 사진이 **있는가**만 본다 — 판정 PASS 라 다시 그림도 안 돈다

한 번은 일화, 세 번은 패턴. 킷에 가지를 **새로** 넣는 것은 여러 실행 기록에서 **반복 확인된 실패**(서로 다른 실행 3곳 이상), 또는 원인과 고칠 법이 **이미 알려진 실패 축**뿐이다. 위 표의 `image_dead`(두 실행) · `image_spec`(게이트 입력 한 번)은 이 기준을 다 채우기 전에 킷 저자가 **보기로** 넣어 둔 것이다. 내가 STEP 6 에서 만든 가지는 내 킷에만 둔다.

인계본 7개를 그대로 받는다 — 잠긴 것은 계약 파일 자체다

루프 결과를 인계본 자리에 넣는다

```
loop_starter_kit/output/graph/  → 통째로 덮어쓴다 →  graph_starter_kit/graph/
   00_START_HERE    01_HANDOFF       02_USER_REQUEST
   03_CONTRACT      04_RUN_EVIDENCE  05_STATE_SEED
   research_data.json

PS> Copy-Item ..\loop_starter_kit\output\graph\* .\graph\ -Recurse -Force
PS> python scripts\check_graph_readiness.py
```

킷의 `graph/` 에는 **베트남 예시본**이 이미 들어 있다. 본인 루프 결과가 있으면 그 위에 덮어쓴다.

첫 실행은 `NOT_READY` (exit 2) 가 정상이다. STEP 을 아직 하나도 안 했기 때문이다.

루프 A 를 돌리기 전이면(`output/research_data.json` 이 없으면) `research_data.json` 이 빠져 **6개**만 생긴다 — 그때는 **덮어쓰지 말고** 킷의 예시 인계본으로 진행한다. 섞으면 나라·일수가 다를 때는 준비도 검사가 막고, 같으면(베트남 5일) 개수 차이가 루프 A 게이트의 SHORT 로 나타난다.

잠긴 것은 숫자가 아니라 `graph/03_CONTRACT.json` 파일 자체다. 검사기 3개는 루프 킷 파일과 **바이트 단위로 같다.** 계약을 그래프용으로 "개선"하는 순간 두 판을 같은 표에 못 올린다.

STEP 은 대화로 진행한다 — Claude 가 결정 2~3개를 A/B/C 와 권장으로 되묻는다

예시 대화 — STEP 3 (형식)

```
나      : STEP 3 진행해줘

Claude : STEP 3 은 라우터가 무엇을 보고 다음 노드를
         고를지 정합니다. 이미 정해 둔 것 — 라우터 4개의
         배선 · `킷 고정 ▼` 처방 가지 분기
         (진단명을 먼저, 판정을 다음에, 상한은 탈출구로)

         질문 1  G-라우터판단 — 무엇을 보고 고를까요?
           A 자유 문장을 직접 본다
           B 허용값이 정해진 필드만 본다   ← 권장
           C LLM 에게 노드 이름을 고르게
         질문 2  G-상한처리 — 상한에 걸리면?
           A 그냥 멈춘다
           B 게이트로 보내 사람이 정한다   ← 권장
           C 상한 없이 계속 돈다

나      : 둘 다 B

Claude : 권장값이라 코드는 그대로입니다.
         검사 check_graph_shape.py → 12행 OK
              validate_specs.py --step 3 → 판정 PASS
         다음: "STEP 4 진행해줘"
```

STEP 카드 읽는 법

칸	무엇을 보나
이 STEP 한 줄	무엇을 정하나 · 안 정하면 무엇이 깨지나
말할 명령	Claude 에게 칠 한 줄과 결정 개수
바뀌는 코드	권장값이면 대개 없음 · 다른 값이면 그 파일
확인	칠 명령 → 기대 출력 · exit
틀리면	무엇이 멈추거나 빨간불이 되나

STEP 문서(`specs/0N_*.md`)는 8구역이다. **고민은 읽기 4(질문)에서만** 하고, 읽기 7 은 무엇이 바뀌는지(코드·파일)를, 읽기 8 이 완료 확인과 다음 명령을 알려 준다. "그냥 권장값으로" 라고 해도 된다 — 그래도 **왜 그게 권장인지 한 줄**은 듣는다.

루프의 조건 7가지가 그래프의 STEP 1~5 로 매칭 — 확인해야하는 결정 13개

루프의 조건	그래프에서	어디
② 관측 표면 · ③ 충족 기준	**그대로**	`03_CONTRACT.json` 잠김
⑤ 판정 독립	**그대로** (검사기 3개)	`scripts/check_*.py` 잠김
⑦ 종료 선언	**그대로** (네 단계)	`01_HANDOFF.md` 잠김
① 시작 상태	→ **상태 스키마**	STEP 1
④ 행동 경계	→ **노드 경계**	STEP 2
⑥ 반복 규칙	→ **간선 조건**	STEP 3
—	**병렬** (신규)	STEP 4
—	**체크포인트 · 사람 개입** (신규)	STEP 5
—	**진단 · 처방 가지** (신규) (킷 고정)	가지 추가만 STEP 6 (조건부)

내가 답하는 13개 · 권장

```
STEP 1  G-상태범위        B   무엇을 담나
        G-원본보관        B   원문을 어디에
        G-판정필드        A   판정을 남기나
STEP 2  G-노드개수        B   몇 개로
        G-검사기노드화    A   노드로 넣나
        G-실행주체        B   어디를 LLM 에
STEP 3  G-라우터판단      B   뭘 보고 고르나
        G-상한처리        B   상한에 걸리면
STEP 4  G-병렬범위        B   어디까지 나란히
        G-병렬충돌        C   같은 칸에 동시에
STEP 5  G-체크포인터      A   무엇으로 저장
        G-게이트위치      B   어디서 멈추나
        G-게이트선택지    B   무엇을 보여 주나
```

권장값을 고르면 코드는 그대로다. 다른 값을 고르면 카드의 **바뀌는 코드** 자리를 고치고 다시 잰다. STEP 6 에는 A/B/C 결정이 없다 — 결정은 13개로 고정이다.

STEP 1 · 상태 — 노드끼리는 맥락을 보지 않고, 상태만을 전달한다.

지도 ▸ **전체** (모든 노드가 같은 상태를 읽고 쓴다) | `specs/01_STATE_SCHEMA.md` · 대상 `travel_graph/state.py`

이 STEP 한 줄	경로 · 판정 · 제어만 칸에 담고 본문은 파일에 둔다. 안 정하면 라우터가 볼 값이 없거나(판정 없음), 상태가 노드마다 통째로 저장돼 재개가 느려진다
말할 명령	`STEP 1 진행해줘` → Claude 가 결정 3개를 묻는다
바뀌는 코드	권장값이면 없음 · 다른 값이면 `state.py` 필드
확인	`python scripts/validate_specs.py --step 1` → 판정 PASS · `check_graph_readiness.py` → NOT_READY (exit 2, 정상 — STEP 이 남았다)
틀리면	원문을 담으면 저장 · 재개가 노드 수만큼 곱해진다 · 판정 칸이 없으면 상한을 비교할 수 없다

```
# 좋은 State — 경로 · 판정 · 제어          # 나쁜 State — 본문 · 통째 · 임시
info_verdict   "SHORT"                      raw_html      "<html>…30만자"
failure_kind   "image_dead"                 everything    {...}
html_path      "output/travel_site.html"    result1, result2
loop_a_iter    2                            temp_result
```

STEP 1 결정 — 권장 B · B · A 면 `state.py` 는 그대로다

ID	질문	A	B	C	권장 · 이유
`G-상태범위`	무엇을 상태에 담나?	판정에 필요한 것만	**A + 제어 필드**	수집 원문까지 전부	**B** — A 면 상한을 비교할 `loop_a_iter` 가 없다 · C 면 저장이 노드 수만큼 곱해진다
`G-원본보관`	수집 원문을 어디에 두나?	상태 안에	**파일에 두고 경로만 상태에**	둘 다	**B** — 상태는 노드가 끝날 때마다 저장된다
`G-판정필드`	검사 결과를 상태에 남기나?	**남긴다** (`*_verdict`)	라우터가 그때그때 검사기를 돌린다	안 남긴다	**A** — B 면 라우터가 부작용을 갖는다. 라우터는 경로만 고른다

> **판정 칸과 진단명 칸을 둘 다 둔다.** `info_verdict` 는 검사기가, `failure_kind` 는 진단이 쓴다 — 라우터가 볼 수 있는 유일한 곳이다. 진단 · 처방 가지 칸 6개는 **킷 고정**이다.

입력 · 출력 스키마를 따로 둔다. Studio 폼에는 `TravelInput` **5칸**만 그려진다 (`inject` 는 실패 재현 칸 — 평소 비운다). 상태 전체는 35칸이다.

STEP 2 · 노드 — 루프의 작업 8개가 노드 14개가 된다, 늘어난 이유를 읽고 승인한다

지도 ▸ **루프 골격 전체** | `specs/02_NODE_BOUNDARY.md` · 대상 없음 (읽고 승인)

이 STEP 한 줄	검사기 · 게이트 · 재수집이 **노드로 선다.** 안 정하면 라우터가 판정을 못 보고(검사기가 노드가 아님), 재수집이 되돌아갈 곳이 없다
말할 명령	`STEP 2 진행해줘` → 결정 3개
바뀌는 코드	무엇을 골라도 없음 — 노드 14개는 `travel_graph/build.py` 의 `SPINE` 에 이미 있다 (정본 이름은 `scripts/check_graph_shape.py` 의 `SPINE_NAMES` 가 쥐고, `SPINE` 은 그것과 같아야 한다)
확인	`python scripts/validate_specs.py --step 2` → 판정 PASS
틀리면	C(16개)는 책임 · 재시도 전략이 같은 것을 나눈다 — 명소 · 일정은 노드가 아니라 **호출**을 나눈 것이다

어디서 온 노드	무엇	개수
루프 **작업**이 노드로	`read_request` `collect_country` `select_cities` `collect_city` `build_page` `quality_pass`	6
루프에 **없던** 노드	`check_information` `check_page` `check_quality` · `human_gate_a` `human_gate_b` · `repair_city` `repair_country` `rebuilder`	8
노드가 **아니게** 된 것	`TASK-기능점검` → 검사 → ◇진단 → 라우터 (간선)	—
다른 노드에 **합쳐진** 것	`TASK-일정수집` → `collect_city` 안에서 `claude -p` 두 번	—

STEP 2 결정 — 검사기는 노드로, 판정은 결정론 코드로, LLM 은 모으고 고치는 자리에만

ID	질문	A	B	C	권장 · 이유
`G-노드개수`	노드를 몇 개로 쪼개나?	루프와 1:1 (8개)	**킷 매핑 (14개)**	더 세분 (16개)	**B** — A 면 검사기가 노드가 아니어서 라우터가 판정을 못 본다
`G-검사기노드화`	검사기를 노드로 넣나?	**노드로 넣는다**	라우터 안에서 검사기를 호출	그래프 밖에서 사람이	**A** — B 면 라우터가 부작용을 갖는다
`G-실행주체`	어느 노드를 LLM 에 맡기나?	전부 LLM	**표대로 섞는다**	전부 결정론	**B** — A 면 판정까지 LLM 이 한다 → 자기 채점표를 통과시킨다

실행 주체	노드 (권장 B 의 표)
LLM · 비결정 · 토큰	`collect_*` · `select_cities` · `repair_*` (재수집)
LLM + 결정론	`quality_pass` · `rebuilder` — 고칠 곳은 LLM 이 지목, 고치기·해시·기록은 코드
결정론 코드	`read_request` · `check_*` 3개 · ◇ `diagnose_*` 2개 · ▷ `fix_*` · `test_*` · `build_page`
사람	★ `human_gate_a` · ★ `human_gate_b`

검사기는 "기준을 채웠나"(`SHORT · 부족 12곳`), **진단은 "무엇이 터졌나"**(`이미지 주소가 죽어 있음 – 증상 1곳`) — 둘 다 노드다. **판정을 LLM 에 맡기지 않는다.** 루프 킷에서 이미 겪었다 — 이미지 11장이 전부 죽어 있는데 검사는 `SUCCESS` 를 냈다. 진단도 같다 — 시험지를 풀어 볼 뿐 LLM 을 부르지 않는다. 재수집(`repair_*`)이 LLM 이라는 점이 STEP 6 의 출발점이다: **LLM 이 같은 실패를 되풀이하면** 그 실패는 규칙 가지 후보가 된다.

STEP 3 · 간선 — 루프의 `if` 가 상태를 읽는 라우터가 된다, 순서는 진단명 → 판정 → 상한

지도 ▸ ◇진단 A 뒤 · **route_after_info** | `specs/03_EDGE_CONDITION.md` · 대상 `travel_graph/edges.py`

이 STEP 한 줄	라우터는 상태만 읽고 다음 노드 이름(또는 `Send` 목록)만 돌려준다. 안 정하면 무한 루프가 나거나 경로가 문장 해석에 좌우된다
말할 명령	`STEP 3 진행해줘` → 결정 2개
바뀌는 코드	권장값이면 없음 · 다른 값이면 그 라우터 함수만 (`STEP 3 ▼▼▼` 구간)
확인	`python scripts/check_graph_shape.py` → 12행 OK · `validate_specs.py --step 3` → 판정 PASS
틀리면	자유 문장 라우팅 · 상한 없는 순환 → 구조 검사가 빨간불 (부록 A)

```
def route_after_info(state):                # 킷의 모양 (줄여 씀)
    # ▼ 킷 고정 — 알아본 실패 → 처방 가지 (LLM 재수집보다 먼저)
    if failure_kind in 처방_가지(A) and 처방_한도가_남았다:
        return failure_kind                 # ▷ fix_<이름>
    # ▲
    if info_verdict != "SHORT":               return "gate"
    if loop_a_iter >= loop_a_limit:           return "gate"   # 바퀴 상한 = 탈출구
    if same_error_count >= same_error_limit: return "gate"   # 진전이 없다
    # 남은 것만 LLM 재수집 — 다친 도시 · 국가 소개만 Send
    return fan_out_repairs(state) or "gate"
```

STEP 3 결정 — 허용값이 정해진 필드만 보고, 상한에 걸리면 사람에게 넘긴다

ID	질문	A	B	C	권장 · 이유
`G-라우터판단`	라우터가 무엇을 보고 고르나?	자유 문장을 직접 본다	**허용값이 정해진 필드만 본다**	LLM 에게 노드 이름을 고르게	**B** — A 는 문장 해석에 좌우 · C 는 같은 입력에 다른 경로
`G-상한처리`	상한에 걸리면?	그냥 멈춘다 (END)	**게이트로 보내 사람이 정한다**	상한 없이 계속 돈다	**B** — A 는 못 채운 칸을 남긴 채 조용히 끝난다 · C 는 무한 루프

문장으로 라우팅하지 않는다. 아래처럼 쓰면 "**더** 모으지 말고 그냥 진행" 에서 '더'가 걸려 재수집으로 간다. 증상 문장(`failure_evidence`)도 같다 — 라우터는 **이름표**(`failure_kind`)만 본다.

```
note = state.get("gate_note")
if "더" in note:
    return "repair"
```

`킷 고정 ▾` **구간은 지우지 않는다.** 처방 가지 분기를 지우면 실패를 재현해도 가지가 영영 안 켜진다. 왜 진단명이 판정보다 먼저인가 — 가지가 보는 결함(깨진 글자 · 원본 크기 사진)은 **검사기가 세지 않는다.** 판정이 PASS 여도 있다.

STEP 4 · 병렬 — reducer 를 떼어 그래프가 멈추는 것을 먼저 보고, 합치는 규칙을 고른다

지도 ▸ 루프 A · **수집 fan-out · 재수집 fan-out** | `specs/04_PARALLEL.md` · 대상 `travel_graph/state.py`

이 STEP 한 줄	정적 간선 여러 개는 **병렬**이다 (if/else 가 아니다). 같은 걸음에 여러 갈래가 같은 칸을 쓰면 reducer 가 합친다. 안 정하면 그래프가 그 자리에서 멈춘다
말할 명령	`STEP 4 진행해줘` → Claude 가 reducer 를 잠깐 떼고 돌려 보게 한 뒤 결정 2개를 묻는다
바뀌는 코드	떼었다 되붙이면 끝 — reducer 3줄 · 함수 2개
확인	`python scripts/run_once.py --mode replay --auto proceed` → 게이트 B 에서 스스로 끝 · `python scripts/verify_all.py --skip-quality` → exit 0
틀리면	`InvalidUpdateError` 로 멈춘다 — 구조 검사가 돌리기 전에 잡는다

`history` 의 reducer 를 떼고 `run_once.py --mode replay --auto stop` 을 돌리면 — 메시지 그대로

```
langgraph.errors.InvalidUpdateError: At key 'history': Can receive only one value per step.
Use an Annotated key to handle multiple values.
```

합류는 리스트로 쓴다. `add_edge(["collect_country", "collect_city"], "check_information")` — 두 줄로 나눠 쓰면 각각이 방아쇠가 되어 뒤쪽 사슬이 두 번 돈다. 검사기 → 진단이 **정적 간선 하나뿐**인 것도 같은 이유다.

STEP 4 결정 — 도시별로 나란히 돌리고, 같은 칸은 이름으로 덮어쓴다

ID	질문	A	B	C	권장 · 이유
`G-병렬 범위`	어디까지 나란히 돌리나?	안 한다 (순차)	**도시별 (+ 국가정보와 도시선정)**	도시 + 관광지별	**B** — 병렬 폭은 `select_cities` 가 정한다 (도시 수만큼 `Send`)
`G-병렬 충돌`	같은 칸에 동시에 쓰면 어떻게 합치나?	`operator.add` (이어 붙임)	칸을 도시별로 쪼갠다	**이름 기준 upsert reducer**	**C** — A 는 재수집 한 번이면 도시가 6개 · B 는 도시 수를 실행 중에 알아 불가능

```
history:       Annotated[list[str],  operator.add]     # 노드마다 한 줄 — 이어 붙인다
cities:        Annotated[list[dict], merge_cities]     # 이름으로 덮어쓴다 (upsert)
city_failures: Annotated[list[dict], merge_failures]   # upsert + 고쳐지면 그 줄을 지운다
```

병렬로 못 하는 것 넷 — 이유가 각각 다르다. `build_page` 는 입력이 다 모여야 · `select_cities` 이전은 병렬 폭을 몰라서 · `quality_pass` 는 앞 회차의 출력이 자기 입력이라서 · 처방 가지는 한 번에 진단명 하나라서 (`failure_kind` 한 칸 · reducer 없음). 회차 카운터에는 reducer 를 붙이지 않는다 (부록 B).

STEP 5 · 사람 — 되물음이 멈춤 → 저장 → 재개가 된다

지도 ▸ **★게이트 A · ★게이트 B** | `specs/05_CHECKPOINT.md` · 대상 `travel_graph/nodes.py` 게이트 2개

이 STEP 한 줄	`interrupt()` 가 상태를 저장하고 그래프를 끝낸다. 사람이 답하면 저장된 자리에서 다시 시작한다. 안 정하면 사람 없이 조용히 끝나거나 노드마다 멈춘다
말할 명령	`STEP 5 진행해줘` → 결정 3개
바뀌는 코드	권장값이면 없음
확인	`python scripts/check_graph_readiness.py` → `[READY]` exit 0 → `langgraph dev --no-reload --port 2024`
틀리면	Studio 에서 `compile()` 에 체크포인터를 넘기면 Agent Server 와 싸운다 · `interrupt` 앞에서 파일을 쓰면 재개 때 한 번 더 쓴다

	사람에게 묻는 방식
루프	`[되물음]` → 터미널에 답 → 그 자리에서 이어진다 (프로세스가 살아 있다)
그래프	`interrupt()` → 상태 저장 → 그래프 끝 → 사람이 답 → 저장된 자리에서 재개. `thread_id` 하나 = 한 번의 실행 · 내일 열어도 이어진다 · 재개하면 그 노드는 **첫 줄부터** 다시 돈다

`interrupt` **앞은 멱등해야 한다.** 파일 쓰기 · 카운터 증가는 `interrupt` **뒤**에 둔다. 정성 회차 **안에서는** 멈추지 않는다 — 해시를 찍고 · 고치고 · 다시 찍는 중에 멈추면 사슬이 끊긴다. 그래서 멈춤은 회차 사이(게이트 B)에만 있다.

STEP 5 결정 — 저장은 Agent Server 에 맡기고, 두 곳에서 선택지 · 현황 · 권장을 함께 보여 준다

ID	질문	A	B	C	권장 · 이유
`G-체크포인터`	상태를 무엇으로 저장하나?	**Agent Server 것 (compile 에 안 붙임)**	SQLite 파일	저장 안 함	**A** — B 는 Studio 것과 싸운다 · C 면 `interrupt` 가 돌지 않는다
`G-게이트위치`	어디서 멈추나?	루프 A 끝만	**루프 A 끝 + 루프 B 끝 (2곳)**	노드마다	**B** — C 는 자동화의 의미가 없다
`G-게이트선택지`	무엇을 보여 주나?	자유 문장으로 받는다	**선택지 + 현황 + 권장**	예/아니오	**B** — A 는 라우팅이 문장 해석에 좌우된다

replay 실행의 게이트 A 화면 — 그대로 (일부)

```
① 지금: 루프 A(정보량) · PASS · 2/3바퀴 · 못 채운 칸 0곳 · 같은 오류 반복 0/2
② 왜: 없음
③ 권장: proceed — 못 채운 칸이 없습니다
④ 고르기:
   more      지적된 것만 다시 모읍니다 — 지적이 없는 곳은 호출조차 하지 않습니다
   proceed   부족해도 페이지를 만듭니다 (못 채운 칸은 보고서에 남습니다)
   stop      여기서 멈춥니다 (END)
```

`action` **만 라우팅에 쓴다.** `note` 는 재수집 · `rebuilder` 프롬프트에 실리는 **지시문**이다.

게이트 B 는 **두 번 선다.** 그 `research` 가 루프에 없던 이득이다 — 페이지에서 수집으로 **간선 하나**.

처방 가지가 못 고친 것은 `② 왜` 맨 앞에 `처방 가지가 못 고침 — <title>` 으로 뜬다 (title = 등록부의 가지 제목, 예: `처방 가지가 못 고침 — 이미지 주소가 죽어 있음`). `more` · `research` · `rebuild` 는 처방 한도도 새로 채운다.

Studio 실행

볼 것

1. `collect_country` 와 `select_cities` 가 **동시에**
2. `collect_city` 3개가 **동시에** — 단, 같은 걸음에 돈 `collect_country` (live LLM)가 끝나야 시작된다
3. 검사기 뒤에 ◇ **진단이 매번** — 판정 다음에 원인
4. 진단 뒤에서 **길 하나**가 골라진다 — 라우터가 상태를 읽은 결과
5. 부족하면 `repair_*` 가 **다친 것만** — 끝난 갈래부터 패널에 쌓인다
6. `check_information` 은 **가장 느린 갈래**가 끝난 뒤에 나타난다 — live 에서 한 갈래는 `claude -p` 1회(타임아웃 600초)와 사진 확인이라 수 분이 걸릴 수 있다
7. `quality_pass` 3개는 **차례로** — 순차가 정상
8. 진단이 **처방 가지**로 보냈나, **재수집(LLM)** 으로 보냈나 — 재수집으로 반복된 것이 STEP 6 후보다

```
python scripts/check_graph_readiness.py   [READY] exit 0
langgraph dev --no-reload --port 2024
  Studio 입력 {} → Submit → 게이트 세 번
python scripts/verify_all.py              exit 0
```

재수집 뒤 `check_information` 이 안 보이면 먼저 기다린다. 확인 순서 — LangSmith 에 열려 있는 `claude -p · repair_*` → `/threads/<tid>/state` 의 `next` → run status → 서버 로그. history 에 `check_information: … 2바 큐` 가 있으면 검사기는 돌았다 — 패널에서 놓친 것이다.

`--no-reload` 는 꼭 붙인다. 실행 중에 `.py` 가 바뀌면(코드 수정 · STEP 6 의 `new_detour.py`) 서버가 다시 떠 돌던 run 이 끊긴다. 새 코드는 서버를 직접 다시 켜야 반영된다.

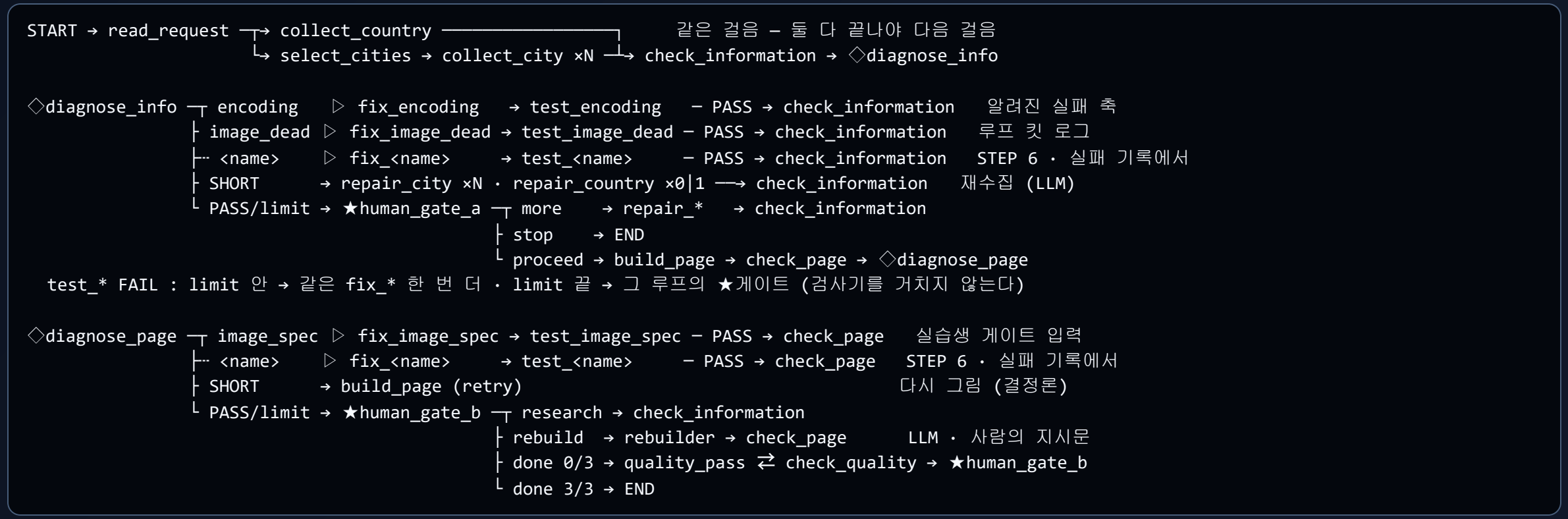
LangSmith 에서 노드별로 본다 — 노드마다 run 하나, 게이트마다 트레이스가 나뉜다

대상	트레이스에 찍히는 모양	읽는 법
노드 22개	루트 `travel` 아래 **노드 이름의 run** — 실행될 때마다 1개	이름 = Studio 그림의 상자
`collect_*` · `repair_*` · `rebuilder` · `quality_pass`	그 아래 `claude -p · <라벨>` → 도구 턴 · `JSON 블록 추출` (라벨 예: `collect_city[하노이]` 명소 · `repair_city[다낭]` · `repair_country` · `정성 판정 1/3회차`)	프롬프트 · 실검색 횟수 · 중단 이유 · 토큰. 폴백은 빨간 run 으로 남는다
라우터 6개	출발 노드의 자식 run (`route_after_info` 는 `diagnose_info` 밑)	갈림길은 라우터 run 의 출력으로 읽는다
`check_information` · `check_page` 반복 방문	방문마다 run 1개 — 이름만으로는 "재수집 뒤" 인지 "가지 복귀" 인지 모른다	메타데이터 `langgraph_step` · 바로 앞 형제 run · history `(처방 가지에서 복귀 …)`
`route_after_detour` ×3	세 가지가 같은 이름을 쓴다	부모 `test_<이름>` 으로 목적지를 읽는다

게이트에서 트레이스가 끝난다. `interrupt` 로 멈추면 그 트레이스가 닫히고, 재개는 **새 루트 트레이스**다. 기본 실행은 트레이스 3~4개로 나뉜다 → LangSmith 의 **Threads** 탭에서 `thread_id` 로 이어 본다.

트레이스는 `langgraph dev` (Studio) 실행에서 남는다 — `langgraph.json` 이 `.env` 의 LangSmith 키를 싣는다. 터미널의 `run_once.py` 는 `.env` 를 읽지 않으므로 **트레이스가 없다.** 그쪽은 history 와 실패 기록으로 본다.

지도 ② — 전체 22노드 개요



순환은 넷 — **안쪽**(재수집 · 다시 그림 · 바퀴 상한 3 · 같은 오류 2번) · **바깥 ★**(사람이 고른다) · **정성**(꼭 3회) · **처방 가지 ▷**(처방 한도 2). 그림에서 뺀 선: `check_quality → build_page`(되돌아감) · `halt → END`. 정본은 `travel_graph/build.py` 맨 위.

STEP 6 — 한 번은 일화, 세 번은 패턴: 실패 기록에 되풀이된 실패가 있을 때만 가지를 연다

`python scripts/failure_report.py` — replay 기본 실행의 끝 (그대로)

```
  ③-2 재수집(LLM)이 반복해서 맡은 원인 — 처방 가지 후보 (같은 입력에 같은 고칠 법이 있으면 가지로)
    (없습니다 — 가지 없이 재수집으로 간 부족이 없거나, 모두 replay 실행입니다)
    (replay 진단 1줄은 세지 않았습니다 — replay 의 재수집은 LLM 이 아니라 인계본을 잘라 씁니다)
======================================================================
  이번 실행의 실패는 이미 처방 가지와 재수집이 맡았습니다 — STEP 6 은 하지 않아도 됩니다
  (live · --all · 여러 교육생의 기록을 모은 폴더에서 후보가 생기면 그때 합니다)
======================================================================
```

진입 조건 — 보고서 ③(규칙표 밖 · 수집 실패 · 가지가 못 맡음)이나 **③-2**(재수집이 맡은 원인)에 `[후보]` 나 `[패턴 — 킷 처방 가지 후보]` 가 있을 때만 — `[후보]` 는 ③ 같은 줄 2번 이상 또는 2실행 이상, ③-2 한 실행 2바퀴 이상 또는 2실행 이상, `[패턴 …]` 은 서로 다른 실행 3곳 이상. 한 번 나온 `[일화]` · `[한 번 — 재수집이 맡으면 된다]` 뿐이면 STEP 6 은 없다. replay 는 ③-2 에서, 재현(inject) 실행은 둘 다에서 세지 않는다 — 후보는 live · `--all` · 모은 기록 폴더에서 생긴다.

후보 (예)	되풀이?	같은 입력 → 같은 고칠 법?	결론
해시를 지어낸 사진 주소	11장 전부 죽음 (루프 킷 로그)	예 — 파일 이름만 뽑아 권장 형태로	**가지** (`image_dead`)
깨진 글자 · 현지 글자 주소	알려진 축	예 — 되읽기 · 퍼센트 인코딩	**가지** (`encoding`)
환율에 출처를 못 붙임	여러 바퀴	아니오 — 조회처와 날짜를 **찾아야** 한다	**재수집에 둔다** — 게이트 `note` 로 방향을 준다
관광지 수가 한 곳 모자람	한 번	—	**재수집이 맡으면 된다**

STEP 6 절차 — 시험지의 빨간불을 먼저 보고, 초록불을 본 뒤 잠근다

```
1 돌린다   live 실행 (또는 --all · 여러 기록을 모은 폴더)
2 읽는다   failure_report.py 의 ③ · ③-2 · history · LangSmith
3 묻는다   되풀이됐나? — 아니오면 끝 (STEP 6 불필요)
4 가른다   같은 입력에 같은 고칠 법이 있나? — 아니면 재수집(LLM)에 둔다
5 만든다   python scripts/new_detour.py <이름> --loop A --title "…"   → 세 조각 · 노드 22 → 24
6 빨간불   시험지 · inject() 를 채우고 run_once.py --mode replay --inject <이름> --auto stop
           → test_<이름>: FAIL
7 초록불   fix() 를 채우고 같은 명령 → test_<이름>: PASS · 1/2회 → check_information (검사기로 복귀)
8 잠근다   inject 없이 '처방 가지 : 타지 않음' · check_graph_shape.py 12행 OK
           · run_graph_regression.py 실패 0 · validate_specs.py --step 6 → 판정 PASS
```

이 STEP 한 줄	재수집(LLM)이 같은 실패를 되풀이했다면 진단이 알아보게 하고, 규칙대로 고치고(fix), 같은 시험지로 잰다(test). **후보가 없으면 건너뛴다**
말할 명령	`STEP 6 진행해줘` → Claude 가 먼저 보고서를 본다. A/B/C 결정은 없고 질문 6개(`specs/06_DETOUR.md` 읽기 4)
바뀌는 코드	`detours/<이름>.py` · `detour_tests/test_<이름>.py` · `registry.py` 한 줄 — `new_detour.py` 가 만든다. 루프 B 가지는 `--loop B` 에 `--auto proceed`
틀리면	시험지를 약하게 만들어 통과시키지 않는다 — `fix()` 를 고친다. 정해진 절차로 안 되면 ★ 게이트로 넘기는 것이 맞는 결과다

정리

같은 계약, 실행 구조만 바뀐다

루프 골격 14 는 잠근다 — 검사 계약 · 검사기 3개는 루프 킷과 같은 파일이다. 그래서 두 판을 같은 검사기로 잰다.

진단은 매번, 처방 가지는 알아본 실패가 났을 때만 — 나머지는 지금처럼 재수집(LLM)이 맡는다.

새 가지는 실패 기록에서 되풀이된 실패가 있을 때만 — 한 번은 일화, 세 번은 패턴. 같은 입력에 같은 고칠 법이 있을 때만 규칙으로 뺀다.